SaralAI

सरल AI · Voice-First Government Services Assistant

Complete Technical & Architecture Reference

Type: Project Documentation · Full Stack Reference

Version: MVP Frontend Complete

Updated: March 2026

Stack: Vite + Vanilla JS · Node.js / FastAPI (planned)

Status: Frontend ✅ Complete · Backend ❌ Planned

Table of Contents

1. Project Identity
2. What Is Done vs Remaining
3. System Architecture
   - 3.1 High-Level Flow
   - 3.2 Data Flow (Client-Side)
   - 3.3 Navigation Flow Diagram
4. Frontend — Full Breakdown
   - 4.1 Project Setup
   - 4.2 Design System & CSS
   - 4.3 Routing
   - 4.4 State Management
   - 4.5 AI Query Engine
   - 4.6 Icons Library
   - 4.7 Components (8 Files)
   - 4.8 Screens (12 Files)
5. Backend — Full Plan
   - 5.1–5.5 Stack · APIs · Services · AI · Data Models
6. Data Layer — Schemes.json
7. Complete User Flow

8. Verification Status

SaralAI — Complete Project Document

Voice-First Government Services Assistant · Technical & Architecture Reference

> *Last updated: March 2026 · Version: MVP Frontend Complete*

Table of Contents

1. Project Identity
1. What Is Done vs Remaining
1. System Architecture
1. Frontend — Full Breakdown
    - Project Setup
    - Design System & CSS
    - Routing
    - State Management
    - AI Query Engine
    - Icons Library
    - Components
    - Screens
1. Backend — Full Plan
    - Technology Stack
    - API Endpoints
    - Services
    - AI Integration
    - Data Models
1. Data Layer — Schemes.json

1. Complete User Flow
1. Verification Status

1. Project Identity

Field	Value
Name	SaralAI (सरल AI)
Meaning	"Simple AI" in Hindi
Tagline	"A simple helper to understand government services"
Type	Voice-first AI assistant for government scheme guidance
Stack	Vite + Vanilla JS (frontend) · Node.js/FastAPI (backend — planned)
Run	`cd c:\Users\siddh\Desktop\SaralAI && npm run dev` → http://localhost:5173

What SaralAI IS vs IS NOT

✅ SaralAI IS	❌ SaralAI IS NOT
A voice-first **explainer** of government schemes	A scheme discovery engine (myScheme does this)
A **simplifier** of complex government language	A government portal replacement
A **step-by-step guide** for applying	An application submission system
A **complementary layer** on top of myScheme	A ChatGPT/Gemini clone
A **client-side, stateless guidance** tool	An eligibility decision maker

2. What Is Done vs Remaining

✅ DONE — Frontend (Current Session)

Phase	What Was Built
Project Foundation	Vite app, `index.html`, `package.json`, `src/styles/index.css` design system
Routing	`src/router.js` — hash-based router for 12 routes
State Management	`src/state.js` — app-wide state with scheme result storage
Icons Library	`src/icons.js` — SVG icon kit (mic, check, arrows, gov, etc.)
Components	All 8 components: Header, Button, MicButton, LanguageCard, AudioPlayer, ProgressIndicator, ChecklistItem, StepCard
All 12 Screens	LandingScreen, LanguageScreen, InteractionModeScreen, SpeakScreen, TypeScreen, ListeningScreen, ProcessingScreen, ExplanationScreen, ClarificationScreen, WhatNextScreen, DocumentsScreen, GuidanceScreen
AI Engine	`src/ai.js` — keyword-based scheme search + intent detection across all 10 MVP schemes
Scheme Data	`src/Schemes.json` — all 10 MVP schemes curated with full details
Data Integration	ProcessingScreen runs real queries → ExplanationScreen shows real scheme data → DocumentsScreen pulls real documents → GuidanceScreen shows real steps
Voice Input	Web Speech API integrated into SpeakScreen + ListeningScreen
Clarification	ClarificationScreen shows top 4 matching scheme cards when query is unclear
WhatNext	Fully dynamic summary with real scheme name, category, and official portal link

❌ REMAINING — Backend & Deployment

Phase	What Needs Building
Backend API	Node.js/FastAPI endpoints for `/api/query`, `/api/scheme`, `/api/intent`
Real STT	Google Cloud Speech or Azure STT for Indian language accuracy
Real TTS	Google TTS or Azure TTS for voice playback of explanations
LLM Integration	Gemini/OpenAI API for advanced simplification prompts
Database	PostgreSQL for scheme data (currently client-side JSON)
PWA	Service worker, offline mode, installable
Deployment	Vercel/Railway frontend, Cloud Run backend
Multi-language UI	Hindi/regional language text in all screens
Analytics	Privacy-safe usage analytics

3. System Architecture

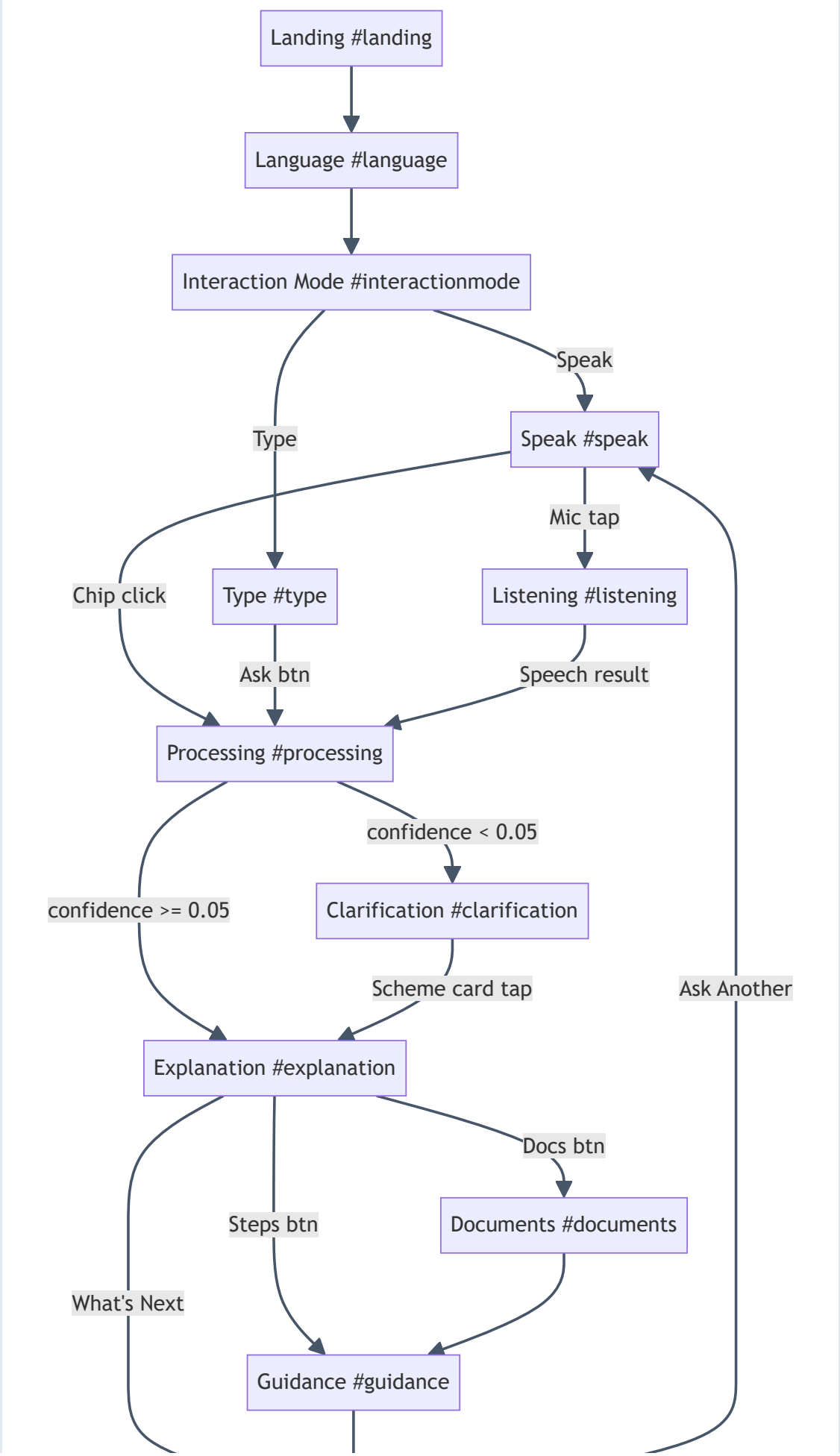
High-Level Flow

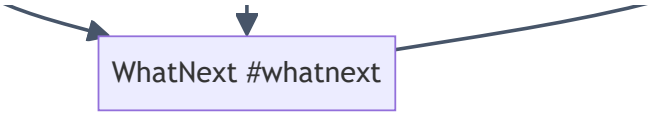
```
User (Voice/Text)
       |
       ▼
┌──────────────────────┐
|    FRONTEND (Vite App)   |
|  http://localhost:5173   |
|                          |
|  ┌──────────────────┐ |
|  |  src/router.js      | |  ← Hash router (#landing, #speak, etc.)
|  |  src/state.js       | |  ← Shared state (query, scheme, language)
|  |  src/ai.js          | |  ← Client-side AI engine
|  |  src/Schemes.json   | |  ← Curated scheme knowledge base
|  |  src/screens/       | |  ← 12 screen modules
|  |  src/components/    | |  ← 8 reusable UI components
|  └──────────────────┘ |
└──────────────────────┘
       |
       ▼ (PLANNED — not yet built)
┌──────────────────────┐
|    BACKEND API           |
|  (Node.js / FastAPI)     |
|                          |
|  POST /api/query         |
|  POST /api/speech-to-text|
|  POST /api/text-to-speech|
|  GET  /api/scheme/:id    |
|  POST /api/intent        |
└──────────────────────┘
       |
       ▼
┌──────────────────────┐
|  EXTERNAL SERVICES       |
|  ├── Google STT API      |
|  ├── Google TTS API      |
|  ├── Gemini/OpenAI LLM   |
|  └── PostgreSQL DB       |
└──────────────────────┘
```

Data Flow (Current — Client-Side)

```
User types query (TypeScreen)
         |
         ▼
setState({ currentQuery })           [src/state.js]
         |
navigate('processing')               [src/router.js]
         |
         ▼
processQuery(query)                  [src/ai.js]
  ├── searchSchemes(query)
  |     └── scores all 10 schemes by keyword overlap
  |         returns ranked [{scheme, confidence}]
  ├── detectIntent(query)
  |     └── classifies as OVERVIEW/ELIGIBILITY/DOCUMENTS/STEPS
  └── generateExplanation(scheme, intent)
        └── returns structured explanation object
         |
         ▼
confidence < 0.05?
  ├── YES → navigate('clarification')   shows top 4 scheme cards
  └── NO  → setState({ currentScheme, currentExplanation, currentIntent })
              navigate('explanation')
                   |
                   ├── Documents btn → navigate('documents')
                   |     pulls state.currentScheme.required_documents
                   └── Steps btn → navigate('guidance')
                         pulls state.currentScheme.application_process.steps
```

Navigation Flow


Landing #landing
Language #language
Interaction Mode #interactionmode
Speak
Type
Speak #speak
Mic tap
Chip click
Type #type
Listening #listening
Ask btn
Speech result
Processing #processing
confidence < 0.05
confidence >= 0.05
Clarification #clarification
Scheme card tap
Ask Another
Explanation #explanation
Docs btn
Steps btn
Documents #documents
What's Next
Guidance #guidance



4. Frontend — Full Breakdown

4.1 Project Setup

`index.html` — Single HTML entry point

```
<div id="app"></div>
<script type="module" src="/src/main.js"></script>
```

- `#app` is the mount point for all screens
- `type="module"` enables ES module imports across all JS files

`package.json` — Minimal deps

```
{ "scripts": { "dev": "vite", "build": "vite build" }, "devDependencies": { "vit
e": "^7.2.4" } }
```

No React, no framework. Pure vanilla JS with Vite for bundling. Vite handles hot module reload, ES module bundling, fast dev server.

`src/main.js` — App entry point (452 lines)

- Imports all 12 screens and 8 components
- Calls `injectStyles()` — collects all CSS exported from screen/component files and appends to
- Calls `initRouter()` — starts hash-based routing
- Defines global `window.SaralAI.showToast()`, `showLoading()`, `hideLoading()` utilities

- Defines shared animation CSS: screen transitions, sound wave animation, skeleton loading, glassmorphism, stagger effects

---

4.2 Design System — `src/styles/index.css`

All CSS variables defined in `:root`. Every value referenced as CSS custom property.

Color Palette (Government Trust)

```
--color-primary: #2563EB        /* Government blue */
--color-primary-dark: #1D4ED8
--color-primary-bg: rgba(37,99,235,0.08)
--color-bg: #F7F8FA             /* Light page background */
--color-bg-card: #FFFFFF        /* Card backgrounds */
--color-text-primary: #1A1A2E
--color-text-secondary: #64748B
--color-text-muted: #94A3B8
--color-success: #10B981
--color-error: #EF4444
```

Typography Scale

```
--font-family: 'Inter', -apple-system, sans-serif
--font-size-xs: 0.75rem  (12px)
--font-size-sm: 0.875rem (14px)
--font-size-base: 1rem   (16px)
--font-size-lg: 1.125rem (18px)
--font-size-xl: 1.25rem  (20px)
--font-size-2xl: 1.5rem  (24px)
--font-size-3xl: 1.875rem (30px)
```

Spacing Scale: `--space-1` (4px) → `--space-20` (80px)

Border Radius: `--radius-sm` (6px) → `--radius-full` (9999px)

Animation keyframes defined: `fadeIn`, `slideUp`, `slideDown`, `scaleIn`, `bounce`, `wave`, `shimmer`

Utility classes: `.container`, `.screen`, `.screen-content`, `.screen-center`, `.card`, `.card-interactive`, `.card-selected`, `.btn`, `.btn-primary/secondary/ghost/dark/lg/sm`, `.icon/icon-sm/icon-lg/icon-xl`, `.animate-fadeIn/slideUp/etc.`

4.3 Routing — `src/router.js`

How it works: Hash-based routing using `window.location.hash`

```
// All 12 routes
const routes = {
  landing, language, interactionmode, speak, type,
  listening, processing, explanation, guidance,
  documents, clarification, whatnext
}

// Navigation
function navigate(route) {
  window.location.hash = route;  // triggers hashchange event
}

// On hashchange → calls onRouteChangeCallback
// main.js registers: onRouteChange(route => renderScreen(route))
// renderScreen adds exit animation, then calls screen.render() + screen.init()
```

Key design decisions:

- Hash router = no server-side routing needed, works on any static host
- Each screen is rendered as an HTML string, not a real DOM tree (no virtual DOM)
- `exit animation (200ms)` → clear `#app.innerHTML` → render new screen string → call `init()` for event bindings

---

4.4 State Management — `src/state.js`

Flat singleton object with pub/sub listeners.

```
const state = {
  // Language
  selectedLanguage: 'en',
  languages: [{code, native, english}],  // 8 languages

  // Interaction
  interactionMode: 'voice',   // 'voice' | 'text'
  isListening: false,
  isProcessing: false,

  // Query
  currentQuery: '',           // User's typed/spoken question

  // Step tracking
  currentStep: 1,
  totalSteps: 3,

  // Query Results (set by ProcessingScreen after ai.js runs)
  currentScheme: null,        // Full scheme object from Schemes.json
  currentIntent: null,        // 'OVERVIEW' | 'ELIGIBILITY' | 'DOCUMENTS' | 'STEP
S'
  currentExplanation: null,   // { schemeName, summary, eligibilityPoints, benefi
tPoints, steps, ... }
  topMatches: [],             // All ranked scheme matches (for clarification scr
een)
  queryHistory: []            // All previous queries this session
}
```

API:

- `getState()` → returns shallow copy of state
- `setState(updates)` → merges updates, notifies all subscribers
- `subscribe(fn)` → registers listener, returns unsubscribe fn
- `getSelectedLanguage()` → returns language object for current code
- `setSelectedLanguage(code)` → convenience wrapper

4.5 AI Query Engine — `src/ai.js`

Purpose: Replaces a backend AI service with a pure client-side JavaScript keyword engine. Reads all data from `Schemes.json` at import time.

`searchSchemes(query)` — Scheme Search

Scores all 10 schemes using keyword overlap:

```
// Each scheme has a keyword list like:
SCHEME_KEYWORDS = {
  PM_KISAN: ['kisan', 'farmer', 'kheti', 'pm kisan', 'samman nidhi', ...],
  PM_JAY: ['ayushman', 'health', 'hospital', 'cashless', 'bimari', ...],
  ...
}

// Scoring algorithm:
function scoreScheme(query, schemeId, scheme) {
  let score = 0, totalWeight = 0;
  // 1. Direct keyword match: score += 1 per keyword hit
  // 2. Scheme name word match: score += 0.5
  // 3. Category match: score += 0.3
  return score / totalWeight;  // 0-1 confidence
}
```

Returns `[{scheme, confidence}]` sorted descending.

`detectIntent(query)` — Intent Classification

Four intent buckets with keyword lists:

```
INTENT_KEYWORDS = {
  DOCUMENTS: ['document', 'papers', 'kya chahiye', 'kaagaz', 'aadhaar', ...],
  STEPS: ['apply', 'kaise', 'process', 'karna', 'fill', 'form', ...],
  ELIGIBILITY: ['eligible', 'kaun', 'paatra', 'milega', 'can i apply', ...],
  OVERVIEW: ['what is', 'kya hai', 'explain', 'batao', 'yojana', ...]
}
```

Counts keyword matches per intent bucket → returns highest scoring intent.

`generateExplanation(scheme, intent)` — Response Generator

Reads from scheme object, builds a structured response:

```
return {
  schemeId, schemeName, category, intent,
  summary,              // context-aware lead sentence
  leadPoints,           // primary bullets based on intent
  eligibilityPoints,    // from scheme.eligibility_criteria[].condition
  benefitPoints,        // from scheme.benefits.details[]
  steps,                // from scheme.application_process.steps[]
  documents,            // from scheme.required_documents[]
  confusions,           // from scheme.common_confusions[]
  limitations,          // from scheme.limitations_and_notes[]
  officialSource,       // from scheme.source_information.official_website
  applicationMode,      // from scheme.application_process.mode
  disclaimer: 'This information is for guidance only.'
}
```

`processQuery(query)` — Main Pipeline

```
export function processQuery(query) {
  const ranked = searchSchemes(query);       // search all 10 schemes
  const { scheme, confidence } = ranked[0]; // top match
  if (confidence < 0.05) return { result: null, confidence, topMatches: ranked };
  const intent = detectIntent(query);        // classify intent
  const result = generateExplanation(scheme, intent);
  return { result, confidence, topMatches: ranked };
}
```

Threshold: confidence < 0.05 → go to Clarification screen. ≥ 0.05 → show Explanation.

4.6 Icons Library — `src/icons.js`

Returns inline SVG strings. Used as `${getIcon('mic', 'icon')}` inside HTML template literals.

Available icons: `mic`, `micOff`, `stop`, `waves`, `arrowRight`, `arrowLeft`, `check`, `x`, `info`, `government`, `keyboard`, `translate`, `idCard`, `language`

Each icon is a pure SVG

with `currentColor` stroke, no external files needed.

4.7 Components (8 Files)

`Header.js` — Sticky top bar

- **Props**: `showNav`, `showLanguageToggle`, `showVoiceAssist`, `showBack`
- Shows SaralAI logo (links to #landing) by default
- `showLanguageToggle=true` → shows "English / English" pill → navigates to #language on click
- `showNav=true` → shows nav links (Home, Services, Language, Help)

- `initHeader()` → attaches click listeners to back and language toggle buttons

`Button.js` — Reusable button

- **Props**: `text`, `variant` (primary/secondary/ghost/dark), `size` (default/lg/sm), `icon`, `iconPosition`, `fullWidth`, `id`, `disabled`
- Returns HTML string: `[]`
- Icons rendered inline from `icons.js`

`MicButton.js` — Microphone button

- `MicButton({size, isListening, id})` → large circle button with pulse animation rings
- `MicButtonSmall({id, text})` → compact mic button with label below
- Animated CSS rings expand outward when `isListening=true`

`LanguageCard.js`

- `LanguageCard({code, native, english, selected})` → single language card
- `LanguageGrid(languages, selectedLanguage)` → 2×4 grid of language cards
- Selected state: blue border + checkmark overlay

`AudioPlayer.js`

- `AudioPlayer({title, subtitle, duration, currentTime})` → "Listen to explanation" card
- Shows waveform animation bars, play/pause button, time display
- Currently static (no real audio — TTS integration planned for backend phase)

`ProgressIndicator.js`

- `ProgressIndicator({current, total, label})` → progress bar + step label
- Used in GuidanceScreen to show "Step 2 of 5"
- CSS animated fill bar

`ChecklistItem.js`

- `ChecklistItem({title, subtitle, icon, checked, id})` → tappable document item
- `Checklist(items)` → stacked list of checklist items
- Checkbox animates to filled blue when checked

- Used in DocumentsScreen for required documents list

`StepCard.js`

- `StepCard({number, title, description, icon, active})` → single numbered step card
- `NumberedSteps(stepsArray)` → renders step list
- Used in legacy ExplanationScreen (now inline numbered list used in rewrite)

4.8 Screens (12 Files)

Each screen file exports three things:

1. `ScreenName()` → returns HTML string (pure function, no DOM access)
1. `initScreenName()` → called after render, attaches event listeners
1. `screenNameStyles` → CSS string injected into by `main.js`

`LandingScreen.js` — Route: `#landing`

What it does: Welcome screen. Builds trust, explains purpose in one line.

Key elements:

- Animated concentric ring icon (government building SVG inside blue circle)
- Heading: "A simple helper to understand government services"
- Subtitle: "Voice-first AI assistant"
- Single "Start" button → navigates to `#language`

No state reads, no data dependencies. Pure UI.

`LanguageScreen.js` — Route: `#language`

What it does: Language selection. Ensures user comfort from first interaction.

Key elements:

- Reads `state.languages` (8 languages: en, hi, bn, te, mr, ta, gu, kn)
- Renders `LanguageGrid` component
- On card click: calls `setSelectedLanguage(code)`, adds checkmark, removes previous selection
- "Continue" button → navigates to `#interactionmode`
- Footer hint with mic icon: "You can also say the name of the language"

State written: `selectedLanguage`

`InteractionModeScreen.js` — Route: `#interactionmode`

What it does: Choose between voice (Speak) or keyboard (Type) input.

Key elements:

- Two large card-buttons: "Speak" (primary, blue gradient) and "Type" (secondary, grey)
- "Speak" card has "Recommended" badge
- Click Speak → setState `interactionMode: 'voice'` → navigate `#speak`
- Click Type → setState `interactionMode: 'text'` → navigate `#type`
- Footer: "You can switch between modes anytime"

Auto-proceeds on selection (no extra continue button).

`SpeakScreen.js` — Route: `#speak`

What it does: Primary voice query entry point.

Key elements:

- Large `MicButton` center-stage with pulsing rings
- Example query: "Ghar ke liye sarkari madad kaise milegi?"
- 4 quick example chips (PM Kisan, PMAY, Ayushman, Ration Card) — clicking any chip sets `currentQuery` directly and navigates to `#processing` (bypasses voice)
- "Type instead" link → `#type`

Voice logic:

```
// Mic tap with Speech API support:
recognition = new SpeechRecognition();
recognition.lang = 'hi-IN,en-IN';
window._saralaiRecognition = recognition; // pass to ListeningScreen
recognition.start();
navigate('listening');

// Without Speech API support:
navigate('type');  // graceful fallback
```

`TypeScreen.js` — Route: `#type`

What it does: Keyboard input fallback/alternative.

Key elements:

- `</code> with 500 char limit counter</li>`